Name: Kelvin Nathans A/L Loganathan ID:24005339

Question 1

Why does BFS ignore the distances stored in the graph even though they are available to it?

BFS only counts edges. It explores level by level, so the first time it reaches the goal, the path has the fewest edges. That is optimal only when every edge costs the same. With unequal costs, the path with the fewest edges may not be the cheapest, so BFS has no reason to use the stored distances. Using them would turn the search into something like UCS.

Question 2

Which two lines change when bfs is turned into dfs, and which of them makes DFS order dependent?

The line that removes a state from the frontier changes from a FIFO removal (popleft()) to a LIFO removal (pop()). The line that adds neighbours to the frontier is the one that makes the order depend on the code. A stack explores the most recently added neighbour first, so the order in which neighbours are pushed, which follows the adjacency list order, decides which branch DFS takes.

Question 3

UCS tests for the goal when a state is popped from the heap. What could go wrong if it tested when the state was first generated?

A state can be generated first through an expensive edge and only later reached more cheaply. If the goal test runs at generation, UCS can return the expensive path. Testing at pop time is safe because, once a state is popped, its g-value is final and no cheaper route can exist.

Question 4

Lab1 reports expanded and runtime_ms. Which one should you use to compare BFS and UCS with a classmate's run on a different laptop, and why?

Use expanded. It counts the states the search processed, which is the same on any machine for the same graph and algorithm. runtime_ms depends on hardware, background load, and the Python version, so it cannot be compared across laptops.

Question 5

Which quantity does GBFS ignore that A uses, and which failure in Lab2 does that explain?*

GBFS ignores g, the cost already paid to reach a state, and ranks states only by h. A* uses f = g + h. Because GBFS ignores g, it can commit to a route that looks close to the goal but is more expensive overall. This explains a Lab 2 result where GBFS returns a path whose cost is higher than the optimal one.

Question 6

Frontier states X and Y have (g, h) = (8, 2) and (3, 6). Which does UCS expand first? GBFS? A?*

UCS ranks by g: Y (3) beats X (8), so Y goes first. GBFS ranks by h: X (2) beats Y (6), so X goes first. A* ranks by f = g + h: X has f = 10 and Y has f = 9, so Y goes first.

Question 7

Why must h be expressed in the same units as the edge costs?

A* adds g and h to get f. If h is in different units, such as kilometres when edges are in minutes, the sum has no meaning. Also, h only guarantees optimal paths when it is a lower bound on the true cost in the same units. Otherwise, the ranking of states is wrong.

Question 8

Lab2 computes h by running Dijkstra on the reversed graph from the goal. Why reversed?*

h*(v) is the cheapest cost from v to the goal. Running Dijkstra from the goal on the reversed graph gives the cheapest cost from every node to the goal in a single run. Running it forward would need a separate search from each node.

Question 9

Weighted A with w = 1.5 changed the cost for KUL→KEF but not for SIN→EZE. What property of the SIN→EZE route explains this?*

Weighted A* uses f = g + w·h. Inflating h changes the ranking only when the heuristic misleads the search, meaning there are competing states whose h values are off. If the SIN→EZE route has h values that are exact or close to exact along the optimal path, the inflated ranking still picks the same states in the same order, so the cost stays the same. KUL→KEF probably has a heuristic that underestimates or misjudges some alternatives, so the weighting changes which states get expanded and which path is returned.